

Numbers with Given Digit Sum (Digit DP - Basic Counting)

Problem

Given two integers:

- n = number of digits
- $target$ = required digit sum

Count all **n-digit numbers** whose digits sum to $target$.

Note: First digit cannot be 0.

Pattern Name

```
Digit Construction DP
      +
State DP (Position + Accumulated Value)
```

Also known as the **Basic Digit DP Pattern** (without tight constraint).

Pattern Identification

Whenever the problem says

- Build a number digit by digit
- String character by character
- Position by position
- Maintain current sum/count/product/etc.

Think

```
Recursion
      ↓
Memoization
      ↓
Tabulation
```

Typical state

```
(position, accumulated_state)
```

Examples

```
(position, sum)
(position, xor)
(position, modulo)
(position, previous_digit)
(position, mask)
```

Step 1 — Understanding the Problem

Example

```
n = 2
target = 2
```

Need all **2-digit numbers**

```
10
11
12
...
99
```

whose digit sum is 2.

Answer

```
11
20
```

Count = 2

Step 2 — Brute Force Thinking

Generate every possible n-digit number.

For each

```
calculate digit sum  
  
if sum == target  
    answer++
```

Complexity

10^n

Impossible for large n .

Step 3 — Building the Recursive Logic

Question 1

What are we constructing?

A number

Digit by digit

- - - -

Question 2

What is one decision?

Choose the next digit.

```
1  
2  
3  
...  
9
```

First position

```
1...9
```

Remaining positions

```
0...9
```

Question 3

What information must recursion remember?

Need two things

Current position

```
i
```

Current digit sum

```
cur_sum
```

State becomes

```
dfs(i, cur_sum)
```

Meaning

Number of ways to fill digits from position `i` given current digit sum = `cur_sum`.

Question 4

When is recursion finished?

When every digit has been chosen.

```
i == n
```

Then

```

if cur_sum == target
    return 1
else
    return 0

```

Recursive Tree

Example

```

n=2
target=2

```

```

    / / / ... \
   1 2 3 ... 9
  /|\
0 1 2 ... 9

```

Generated numbers

```

10
11
12
...
19

20
21
...

```

Building Choices

This is where most people get stuck.

Always ask

What values can I legally place here?

First digit

Cannot be 0

Choices

1...9

Remaining digits

0...9

Hence

```
start = 1 if i == 0 else 0
for digit in range(start, 10):
```

Nothing magical.

It simply represents

Legal choices.

Recursive Code

```
def dfs(i, cur_sum):
    if cur_sum > target:
        return 0

    if i == n:
        return 1 if cur_sum == target else 0

    ans = 0

    start = 1 if i == 0 else 0

    for digit in range(start, 10):
```

```
ans += dfs(i + 1, cur_sum + digit)

return ans
```

Why Pruning Works

If

```
cur_sum > target
```

No future digit can reduce the sum.

Digits are

```
0...9
```

Only increase the sum.

So immediately

```
return 0
```

Complexity

Without DP

```
10^n
```

Memoization

Step 1

Observe recursion state

```
dfs(i, cur_sum)
```

Therefore

```
dp[i][cur_sum]
```

Nothing else is required.

DP Size

Rows

```
0...n
```

```
Total
```

```
n+1
```

Columns

```
0...target
```

```
Total
```

```
target+1
```

```
dp = [[-1]*(target+1) for _ in range(n+1)]
```

Memo Transition

Recursive equation

```
dfs(i, sum)
=
Σ dfs(i+1, sum+digit)
```

Store

```
dp[i][sum]
```

Memo Code

```
memo = [[-1]*(target+1) for _ in range(n+1)]

def dfs(i, cur_sum):
    if cur_sum > target:
        return 0

    if i == n:
        return 1 if cur_sum == target else 0

    if memo[i][cur_sum] != -1:
        return memo[i][cur_sum]

    ans = 0

    start = 1 if i == 0 else 0

    for digit in range(start, 10):
        ans += dfs(i+1, cur_sum+digit)

    memo[i][cur_sum] = ans

    return ans
```

Complexity

States

$O(n \times \text{target})$

Transitions

10

Total

$O(n \times \text{target} \times 10)$

Converting Memo → Tabulation

This is the MOST IMPORTANT PART.

Rule 1

Copy the recursion state.

```
dfs(i, sum)
```

↓

```
dp[i][sum]
```

Rule 2

Copy base case.

Recursion

```
if i==n
```

```
return 1 if sum==target
```

Table

```
dp[n][target]=1
```

Everything else remains 0.

Rule 3

Check dependency direction.

Recurrence

```
dp[i]
```

```
depends on
```

```
dp[i+1]
```

Need future row.

Therefore fill

```
n
↑
n-1
↑
...
0
```

Loop

```
for i in range(n-1, -1, -1):
```

Rule 4

Current sum

Need every possible sum

```
0...target
```

Loop

```
for cur_sum in range(target, -1, -1):
```

Ascending also works because we only use the next row.

Rule 5

Replace recursive call.

Memo

```
dfs(i+1,cur_sum+digit)
```

becomes

```
dp[i+1][cur_sum+digit]
```

Tabulation Code

```
dp = [[0]*(target+1) for _ in range(n+1)]
dp[n][target] = 1
for i in range(n-1,-1,-1):
    start = 1 if i==0 else 0
    for cur_sum in range(target,-1,-1):
        ans = 0
        for digit in range(start,10):
            if cur_sum + digit <= target:
                ans += dp[i+1][cur_sum+digit]
        dp[i][cur_sum] = ans
return dp[0][0]
```

Why Reverse Loop?

Because

Current row

↓

Needs

↓

Next row

`dp[i]`

↓

`dp[i+1]`

Future values must already exist.

Hence

Bottom

↓

Top

DP Table

```

        cur_sum
      0 1 2 3 .... target
i=0
i=1
i=2
...
i=n-1
i=n    0 0 0 ... 1

```

Space Optimization

Observe

Current row
depends only on
Next row

Only one row required.

0(target)

Edge Cases

Impossible sum

target > 9*n

Answer

0

(or -1 depending on problem statement)

Negative sum

Impossible.

n=1

Only digits

1...9

target=0

Possible only if leading zeros are allowed.

Otherwise

No n-digit number exists.

Answer = 0

(except special problem variants)

Pattern Recognition Sheet

Whenever you see

Construct

Number

String

Array

Expression

position by position

AND maintain

Current Sum

Current XOR

Current Mod

Current Product

Current Cost

Think

State DP

(position, accumulated_state)

Template

```
def dfs(position, state):  
    if invalid:  
        return 0  
  
    if position == n:  
        return valid_state  
  
    ans = 0  
  
    for choice in legal_choices:  
        ans += dfs(next_position,  
                    updated_state)  
  
    return ans
```

Memo

```
dp[position][state]
```

Tabulation

Rows

position

Columns

state

Similar Interview Questions

Easy

- Count numbers with given digit sum
- Number of binary strings with k ones
- Count strings with vowel count = k

Medium

- Decode Ways
- Restore IP Addresses
- Count Good Strings
- Target Sum
- Combination Sum IV

Advanced Digit DP

- Count numbers in range
- Sum of digits in range
- Numbers without repeated digits
- Numbers with adjacent difference
- Digit DP with Tight Constraint
- Numbers divisible by K
- Beautiful Integers

Common Mistakes

✗ Allowing first digit = 0

✗ Forgetting

```
cur_sum > target
```

pruning

✗ Wrong DP size

```
Need
```

```
target+1
```

not $9 * n$

when pruning exists.

✗ Wrong loop direction

Always inspect dependency

```
dp[i]
```

```
↓
```

`dp[i+1]`

↓

Reverse loop

Real Interview Quick Sheet

Question asks

↓

Build Number/String Digit-by-Digit?

↓

YES

↓

State =

(position, accumulated_value)

↓

Recursion

↓

Memo

↓

Tabulation

↓

DP dimensions = recursion parameters

↓

Base case = last row

↓

Dependency decides loop direction

↓

Replace recursive call with DP lookup

↓

Done.

The Biggest Lesson from This Problem

Whenever you get stuck while writing recursion, don't think about code first. Ask these four questions in order:

1. What am I constructing?

- A number, string, path, subset, etc.

2. What position am I currently filling?

- This usually becomes the **position/index** state.

3. What extra information must I carry forward?

- Sum, XOR, modulo, previous value, mask, etc.

4. What are the legal choices at this position?

- The answer to this question directly becomes your **for** loop.

If you can answer these four questions, the recursion, memoization, and tabulation almost always follow mechanically.